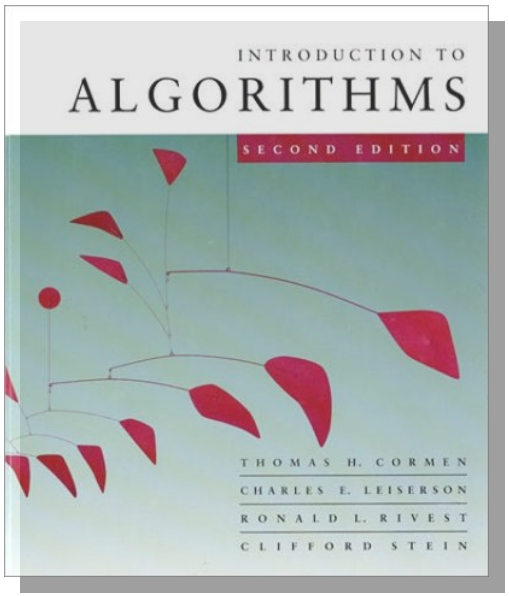
Introduction to Algorithms
6.046J/18.401J



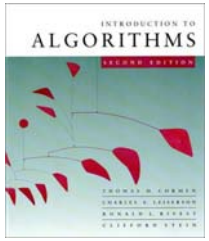
LECTURE 1

Analysis of Algorithms

- Insertion sort
- Asymptotic analysis
- Merge sort
- Recurrences

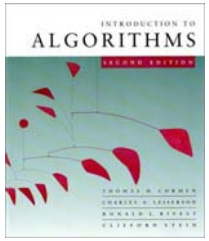
Prof. Charles E. Leiserson

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Course information

1. Staff
2. Distance learning
3. Prerequisites
4. Lectures
5. Recitations
6. Handouts
7. Textbook
8. Course website
9. Extra help
10. Registration
11. Problem sets
12. Describing algorithms
13. Grading policy
14. Collaboration policy



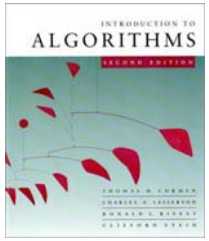
Analysis of algorithms

The theoretical study of computer-program performance and resource usage.

What's more important than performance?

- modularity
- correctness
- maintainability
- functionality
- robustness
- user-friendliness
- programmer time
- simplicity
- extensibility
- reliability

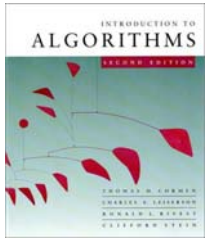
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Why study algorithms and performance?

- Algorithms help us to understand ***scalability***.
- Performance often draws the line between what is feasible and what is impossible.
- Algorithmic mathematics provides a ***language*** for talking about program behavior.
- Performance is the ***currency*** of computing.
- The lessons of program performance generalize to other computing resources.
- Speed is fun!

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



The problem of sorting

Input: sequence $\langle a_1, a_2, \ldots, a_n \rangle$ of numbers.

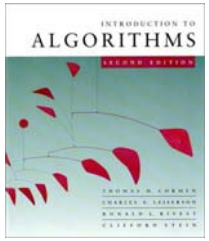
Output: permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ such that $a'_1 \le a'_2 \le \cdots \le a'_n$.

Example:

Input: 8 2 4 9 3 6

Output: 2 3 4 6 8 9

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

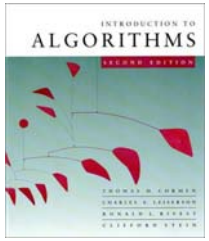


Insertion sort

"pseudocode"

```
INSERTION-SORT (A, n)        ▷ A[1 . . n]
    for j ← 2 to n
        do key ← A[j]
           i ← j – 1
           while i > 0 and A[i] > key
               do A[i+1] ← A[i]
                  i ← i – 1
           A[i+1] = key
```

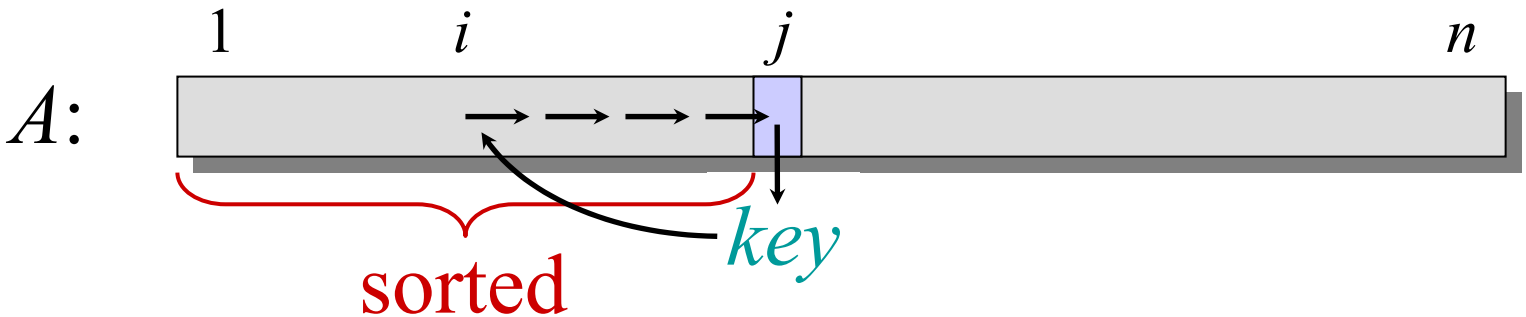

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Insertion sort

"pseudocode"

```
INSERTION-SORT (A, n)        ▷ A[1 . . n]
    for j ← 2 to n
        do key ← A[j]
           i ← j – 1
           while i > 0 and A[i] > key
               do A[i+1] ← A[i]
                  i ← i – 1
           A[i+1] = key
```



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Example of insertion sort

8 2 4 9 3 6

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Example of insertion sort

8 2 4 9 3 6

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Example of insertion sort

8 2 4 9 3 6

2 8 4 9 3 6

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Example of insertion sort

8 2 4 9 3 6

2 8 4 9 3 6

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6
2	4	8	9	3	6

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6
2	4	8	9	3	6

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6
2	4	8	9	3	6
2	3	4	8	9	6

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6
2	4	8	9	3	6
2	3	4	8	9	6

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Example of insertion sort

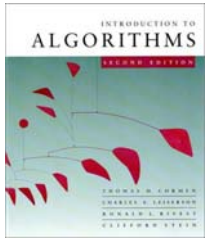
8	2	4	9	3	6	
2	8	4	9	3	6	
2	4	8	9	3	6	
2	4	8	9	3	6	
2	3	4	8	9	6	
2	3	4	6	8	9	*done*

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Running time

- The running time depends on the input: an already sorted sequence is easier to sort.
- Parameterize the running time by the size of the input, since short sequences are easier to sort than long ones.
- Generally, we seek upper bounds on the running time, because everybody likes a guarantee.

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Kinds of analyses

Worst-case: (usually)

- $T(n)$ = maximum time of algorithm on any input of size n.

Average-case: (sometimes)

- $T(n)$ = expected time of algorithm over all inputs of size n.
- Need assumption of statistical distribution of inputs.

Best-case: (bogus)

- Cheat with a slow algorithm that works fast on *some* input.

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Machine-independent time

What is insertion sort's worst-case time?

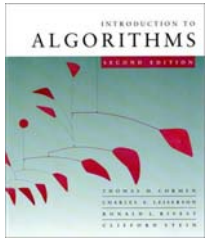
- It depends on the speed of our computer:
  - relative speed (on the same machine),
  - absolute speed (on different machines).

BIG IDEA:

- Ignore machine-dependent constants.
- Look at ***growth*** of $T(n)$ as $n \to \infty$.

"Asymptotic Analysis"

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Θ-notation

Math:

$\Theta(g(n)) = \{ f(n) :$ there exist positive constants c_1, c_2, and n_0 such that $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$ for all $n \ge n_0 \}$

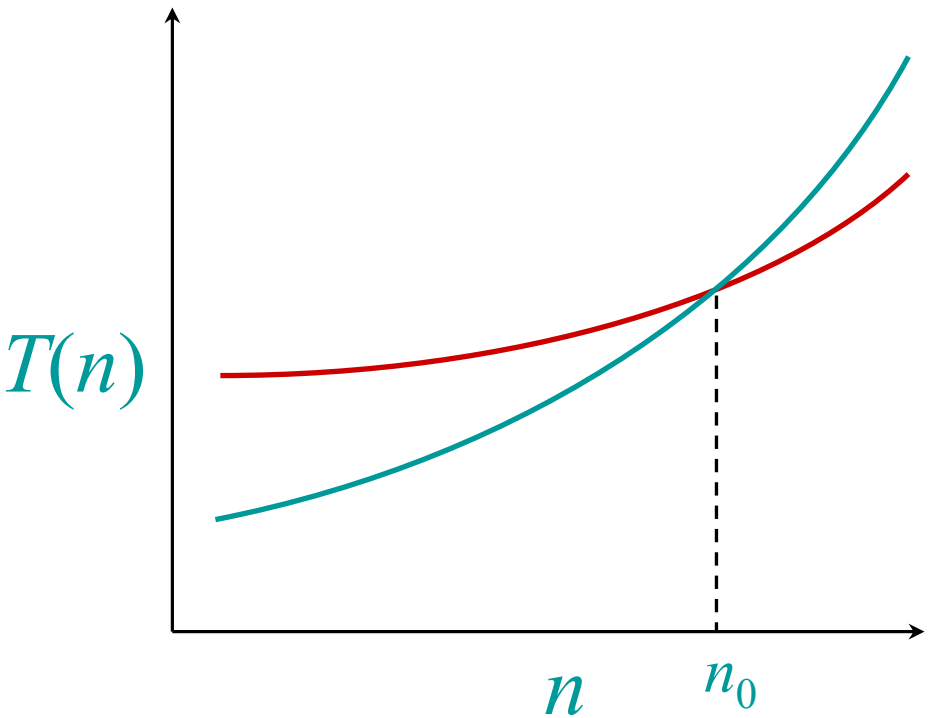
Engineering:

- Drop low-order terms; ignore leading constants.
- Example: $3n^3 + 90n^2 - 5n + 6046 = \Theta(n^3)$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Asymptotic performance

When n gets large enough, a $\Theta(n^2)$ algorithm *always* beats a $\Theta(n^3)$ algorithm.



- We shouldn't ignore asymptotically slower algorithms, however.
- Real-world design situations often call for a careful balancing of engineering objectives.
- Asymptotic analysis is a useful tool to help to structure our thinking.

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson